

LedgerLM — Design Document

Version: 1.1 · **Date:** 2026-07-03 · **Status:** Approved for build · (1.1: *project skill harness added* — §8, §12, D13) **Role of this document:** This is the build contract. Every build session (Claude Code) starts by reading this file, executes exactly one phase, and stops at that phase's gate. It supersedes all prior planning notes. Changes to decisions are recorded in the Decision Log (§13) and committed like any other change.

1. What we are building

LedgerLM is a local-first Python library and local dashboard that instruments every LLM API call an application makes, and records each call as a row in a local ledger: provider, model, four normalized token buckets (uncached input, cache reads, cache writes, output), USD cost computed from provider-returned usage against a local price table, latency, status, and attribution tags (project / feature / env / run_id / customer, plus arbitrary extras). On top of the ledger sit a CLI, a dashboard (spend over time, breakdowns, top calls), alerts (budget + spike → webhook), an arithmetic-only optimizer report, and CSV export.

Instrumentation is two lines: wrap the SDK client, open a tag scope. The SDK response is returned unmodified, and recording can never break the host application.

Primary user: the author — a Python developer running a multi-agent blog-automation network with heavy Anthropic and OpenAI usage, building this to see and cut his own spend.

Secondary user: Python developers generally, via public release. Launch channel: the Bench Notes blog (honest AI-tool cost breakdowns), whose audience is exactly the target user.

Everything in v0 runs locally, self-hosted, single user. No account, no external service, no data leaving the machine.

2. Why

Developers running LLM apps cannot see, in one place and near-real-time, what they are spending, on which models, for which part of their application. Provider dashboards are siloed (one per vendor), delayed, and blind to application structure — they cannot say what a feature, project, or customer costs. Spend spikes surface on the invoice, weeks too late to act.

Existing tools do not close this gap:

- **Helicone** is an observability/analytics product that wants to sit in your request path, primarily as a hosted proxy. It adds a third party between you and your provider and is organized around request logging, not local cost accounting.
- **Langfuse** is a self-hostable but full-blown LLM engineering platform — tracing, evals, prompt management — that has grown into a multi-service deployment. Cost is a feature hanging off traces, not the organizing object, and running it is a project in itself.
- **LiteLLM** is the closest overlap: its proxy tracks spend and budgets. But it is a routing/infrastructure layer you deploy as a server, with spend keyed to API keys and teams — not to in-code business dimensions — and adopting it means adopting its gateway.

The wedge: nobody offers *pip install, wrap your client, get a trustworthy local cost ledger mapped to your features* — no server, no account, no third party in the request path. Local-first plus cost-attribution-as-the-primary-object is unserved.

Honest caveat: LiteLLM could plausibly move toward this space, so speed matters, and so does the differentiator it cannot easily copy: a brand built on accuracy (which fits the Bench Notes launch). The entire value of this tool is that its numbers are trustworthy; every design decision below bends toward that.

3. Product principles (non-negotiable, enforced in every phase)

1. **Cost only from provider-returned usage.** Never estimate cost from prompt text or token-count heuristics.
2. **Never fabricate.** Unknown (provider, model) pricing → the event is recorded with full token counts and `cost_usd = NULL` ("unpriced"), with one warning per model. Never write \$0 for an unknown price. Every surface that shows cost also shows the count of unpriced rows.
3. **Auditable rows.** Every event stores the provider's raw usage object verbatim (`raw_usage` JSON) and the per-Mtok rates applied (`price_snapshot` JSON). Any row's cost is independently recomputable from its own contents.
4. **The recorder never raises.** Instrumentation failures (DB down, disk full, bug) are logged and swallowed; the wrapped call's result is always returned to the caller. LedgerLM must never break or block the host app.
5. **Local-first and offline-capable.** No required external service. Dashboard assets are vendored (no CDNs). Dashboard binds 127.0.0.1 by default. The only network calls LedgerLM itself makes are user-configured webhooks.
6. **Postgres-ready SQLite.** SQLite (WAL mode, `busy_timeout`) is the default store; the schema stays Postgres-compatible; Alembic runs with `render_as_batch=True`.
7. **Light by default.** The `anthropic` and `openai` SDKs are optional extras with lazy imports. The core package (mock provider, CLI, DB) installs and runs with neither.
8. **UTC in storage,** always. Display conversion is a presentation concern.

9. **Honest analysis.** The optimizer makes arithmetic claims ("identical tokens at model X would have cost \$Y"), never quality claims ("model X would have sufficed"). Its outputs are labeled as candidates for experiments, not verdicts.
10. **No prompt content stored.** Only a `prompt_hash` (SHA-256 over a canonical serialization of the request's message content) — enough for repeat-prompt detection, nothing to leak.

4. Scope

v0 (this document's trajectory): Python instrumentation for Anthropic + OpenAI (sync, async, and streaming), the mock provider, cache-aware cost accounting, SQLite persistence with migrations, CLI, local dashboard, webhook alerts, optimizer report, CSV export, PyPI release.

Explicitly out of v0 — noted with the seam each one later plugs into (§10): hosted/multi-tenant SaaS, user auth/billing/team seats, hard budget caps or enforcement, providers beyond Anthropic/OpenAI, Batch API ingestion, Slack/Discord delivery, and a local proxy mode. Nothing in v0 may preclude these.

5. Architecture

your app code

```
| client = ledgerlm.wrap(Anthropic())    with ledgerlm.tags(project=..., feature=...):
```



`wrap()` transparent proxy — all other attrs/methods pass through —► real SDK —► provider API

```
| intercepts known call paths only
```



provider adapter (anthropic | openai | mock)

```
| normalize usage → 4 disjoint token buckets; compute prompt_hash
```



pricing engine (model_prices table) —► cost_usd (Decimal) or NULL (unpriced)



recorder (never raises) —► SQLite ledger (WAL) ◀—— CLI: summary / prices / export



dashboard (FastAPI + HTMX, read-only) — alerts evaluator —► webhook



optimizer report (read-only)

The load-bearing separation: **adapters normalize, the pricing engine prices, the recorder persists**. The recorder consumes normalized call events and does not know or care who produced them — the wrap() proxy is merely its first producer. This is the seam that later admits a local proxy mode, a Batch importer, or a hosted ingestion API without rearchitecting.

Key decisions and their rationale live in the Decision Log (§13); the schema-level consequences are in §6.

6. Data model

llm_events

id	PK	
ts	DATETIME (UTC)	indexed
provider	TEXT	"anthropic" "openai" "mock"
model	TEXT	indexed
status	TEXT	"ok" "error"
error_type	TEXT NULL	exception class name on errors
latency_ms	INTEGER	wall time to completion
first_token_ms	INTEGER NULL	streaming only (added in Phase 1.5)
input_tokens	INTEGER	normalized: UNCACHED input only
output_tokens	INTEGER	
cache_read_tokens	INTEGER NULL	

cache_write_tokens INTEGER NULL

raw_usage JSON provider usage object, verbatim

price_snapshot JSON NULL per-Mtok rates applied at record time

cost_usd NUMERIC(18,10) NULL NULL = unpriced (never a fabricated 0)

prompt_hash TEXT NULL indexed; sha256, content never stored

project TEXT NULL indexed

feature TEXT NULL indexed

env TEXT NULL

run_id TEXT NULL indexed

customer TEXT NULL

tags JSON NOT NULL DEFAULT {} arbitrary extra tags

provider_request_id TEXT NULL

created_at DATETIME (UTC)

composite indexes: (project, ts), (model, ts)

model_prices

id PK

provider + model UNIQUE together

input_per_mtok NUMERIC

output_per_mtok NUMERIC

cache_read_per_mtok NUMERIC NULL

cache_write_per_mtok NUMERIC NULL

currency TEXT DEFAULT "USD"

last_verified DATE NULL honesty marker for seed data

updated_at DATETIME (UTC)

alert_firings (created in Phase 3)

id, rule TEXT, window_start, window_end, observed NUMERIC, threshold NUMERIC,

fired_at DATETIME (UTC), delivered BOOL, response_status INTEGER NULL

Rationale, briefly: token counts are exact integers and the price snapshot travels with the row, so the ledger is self-auditing — SQLite's float-backed NUMERIC storage can never corrupt the truth, because `cost_usd` is derivable from the row itself. The five attribution dimensions are promoted, indexed columns (fast grouping, obvious queries); everything else rides in `tags` JSON, queryable via `json_each` on SQLite and native JSON operators on Postgres. Alert *configuration* lives in a TOML file (local-first, versionable); only firings are persisted, for cooldown/dedupe. Aggregations are computed on read — no rollup tables unless a measured need appears (D10).

7. Tech stack

- **Runtime:** Python 3.11+. Dependencies: `sqlalchemy>=2`, `alembic`, `typer`, `pydantic-settings`, `pyyaml`. Phase 2 adds `fastapi`, `uvicorn`, `jinja2`; Phase 3 adds `httpx` (webhooks).
- **Optional extras:** `[anthropic]`, `[openai]`, `[all]`; dev tooling in `[dev]` (`pytest`, `pytest-asyncio`, `ruff`, `mypy`).
- **Frontend:** server-rendered Jinja2 + HTMX partials + Chart.js, all vendored into `static/` (no CDNs). Chosen because the dashboard is read-mostly-with-filters (HTMX's sweet spot), it keeps the whole product in one language with zero JS build toolchain for a solo backend maintainer, and the JSON endpoints backing the pages preserve the option of a SPA later. React+Vite+Recharts was considered and rejected as a second toolchain to maintain alone for ~15% more polish.
- **Quality:** `ruff` (lint + format), `mypy` on `src` with reasonable strictness (targeted, commented ignores acceptable at SDK boundaries), `pytest` with a hard no-network rule (live-key integration tests exist but are always env-gated and skipped by default).
- **Packaging:** `src` layout, hatchling backend, pip-installable; CI via GitHub Actions (`ruff`, `mypy`, `pytest` on Python 3.11/3.12/3.13).
- **Containers:** Dockerfile + docker-compose for the dashboard. Known caveat: SQLite WAL over Docker Desktop bind mounts (macOS/Windows) is unreliable, so the primary run mode is `ledgerlm dashboard` on the host; compose targets Linux hosts and the future Postgres flavor.

8. Repository layout (final state, all phases)

ledgerlm/

pyproject.toml README.md CHANGELOG.md DESIGN.md CLAUDE.md

alembic.ini .gitignore

.github/workflows/ci.yml

.claude/skills/ # project skill harness — pre-seeded before Phase 0 (§12, D13)

executing-a-phase/SKILL.md

building-provider-adapters/SKILL.md

changing-the-schema/SKILL.md

computing-costs/SKILL.md

Dockerfile docker-compose.yml

data/prices_seed.yaml

src/ledgerlm/

__init__.py # public API: wrap, tags, __version__

config.py # pydantic-settings (env prefix LEDGERLM_)

tagging.py # contextvars tag scope

pricing.py # price lookup + Decimal cost computation

recorder.py # event persistence; never raises

providers/

base.py anthropic.py openai.py mock.py

db/

```
models.py session.py

migrations/      # alembic env + versions

cli.py           # Typer entry point `ledgerlm`

dashboard/       # Phase 2

app.py queries.py

templates/*.html

static/ htmx.min.js chart.umd.js style.css

alerts.py        # Phase 3

optimizer.py     # Phase 3

export.py        # Phase 3

tests/

conftest.py test_normalizers.py test_pricing.py test_tags.py

test_smoke.py test_streaming.py test_dashboard.py test_alerts.py

test_optimizer.py
```

9. Build trajectory

Phases run strictly in order. Each ends at a **gate**. A *hard gate* means: stop, summarize what was built and every deviation from this document, and wait for human review — do not start the next phase. A *soft gate* means: self-verify the exit criteria, commit, and continue in the same session. Any deviation that survives review becomes a new Decision Log row.

Phase 0 — Foundation (init) · soft gate

Goal: a green, empty skeleton so every later phase lands on working rails.

Build:

- `pyproject.toml` (hatchling, src layout, `[project.scripts] ledgerlm = ...`, extras `[anthropic] [openai] [all] [dev]` declared even where not yet used).

- `src/ledgerlm/__init__.py` with `__version__ = "0.0.1"`; ruff + mypy + pytest configuration; `.gitignore`; one trivial import test.
- `.github/workflows/ci.yml`: ruff check, mypy, pytest on Python 3.11/3.12/3.13.
- `CLAUDE.md`: conventions (small conventional commits, no-network tests, never-raise recorder), pointers to this DESIGN.md as the contract and to the skill harness in `.claude/skills/`, and a "current phase" marker updated at every gate.
- `README.md` stub: one-paragraph description + "under construction".
- Confirm the four pre-seeded skills in `.claude/skills/` are discovered and commit them as part of the initial tree.

Exit criteria: `pip install -e ".[dev]"` clean; `ruff check`, `mypy src`, `pytest` all green. Continue directly to Phase 1.

Phase 1 — Core ledger · HARD GATE 1

Goal: wrapped SDK calls become priced, tagged, auditable rows; the CLI can report them.

Public API (design target):

```
import ledgerlm
```

```
from anthropic import Anthropic
```

```
client = ledgerlm.wrap(Anthropic()) # also AsyncAnthropic, OpenAI, AsyncOpenAI
```

```
with ledgerlm.tags(project="blog-net", feature="summarize", run_id=run_id):
```

```
    resp = client.messages.create(model="...", max_tokens=512, messages=[...])
```

resp is the unmodified SDK response; exactly one `lm_events` row was recorded

Build:

- **wrap() transparent proxy.** Intercepts only known call paths — Anthropic `messages.create`; OpenAI `chat.completions.create` and `responses.create` — and passes every other attribute/method through untouched. Detects sync vs async clients and mirrors them. Streaming calls in this phase pass through *unbroken and unrecorded* with a one-time warning ("streaming capture lands in Phase 1.5"). No retry logic anywhere: the SDKs retry internally; LedgerLM records one event per completed call, `status="ok"` or `"error"` (exception class + latency; usage if present).
- **ledgerlm.tags(...):** contextvars-based, nestable, async-safe context manager. Reserved keys `project/feature/env/run_id/customer` map to columns; other kwargs land in the `tags` JSON column. Inner scopes override outer per-key.

- **Usage normalization (correctness-critical).** Four disjoint buckets. OpenAI chat completions: `prompt_tokens` INCLUDES `prompt_tokens_details.cached_tokens` — subtract to get uncached input; `completion_tokens` → output; the Responses API's `input_tokens/input_tokens_details.cached_tokens` have the same subset semantics. Anthropic: `input_tokens` EXCLUDES `cache_creation_input_tokens` and `cache_read_input_tokens` (already disjoint) — map directly. Adapters also compute `prompt_hash` = SHA-256 over a canonical JSON serialization of the ordered message/system content (content never stored).
- **Cost computation.** Decimal arithmetic only: sum over nonzero buckets of $(\text{tokens} / 1_000_000) \times \text{per-Mtok rate}$, quantized to 10 dp. If any rate needed for a nonzero bucket is missing → the whole row is unpriced (NULL) with one warning per (provider, model).
- **Persistence.** SQLAlchemy 2.0 `Mapped[ ]` models per §6 (including `prompt_hash`; excluding `first_token_ms` and `alert_firings`, which arrive with their phases); one initial Alembic migration; SQLite WAL + `busy_timeout`; engine/session factory from config.
- **Config.** pydantic-settings, env prefix `LEDGERLM_`. `db_url` default `sqlite:///~/.ledgerlm/ledgerlm.db` (expanduser, create parent dir) — one shared per-user ledger so multiple apps attribute into the same DB; per-project override via env. `echo_sql` default false.
- **Pricing seed.** `data/prices_seed.yaml` with current mainstream Anthropic + OpenAI models. Verify rates against official pricing pages if web access is available and set `last_verified`; otherwise mark entries clearly UNVERIFIED with `last_verified: null`. A short verified list beats a long guessed one. `ledgerlm init` loads the seed only if `model_prices` is empty; the DB is authoritative thereafter. Seed includes a "mock" model so tests exercise real cost math.
- **Mock provider** ships in the package (`providers/mock.py`): SDK-shaped `MockLLMClient`, deterministic, configurable usage across all four buckets; `wrap()` recognizes it as provider "mock".
- **CLI (Typer):** `init` (upgrade schema + seed-if-empty); `summary [--since 7d|24h|30d] [--by provider|model|project|feature]` showing calls, tokens in/out, cost, and ALWAYS the unpriced-row count; `prices list`; `prices set PROVIDER MODEL --input X --output Y [--cache-read Z --cache-write W]`; `prices backfill` (recompute unpriced rows that now have full rates, writing `price_snapshot` from current rates).
- **README quickstart** (~10 lines) with example `summary` output.

Required tests:

1. Smoke: wrap `MockLLMClient` → nested tag scopes → call → exactly one persisted row; exact Decimal cost asserted against a hand-computed value including cache

buckets; tag columns + JSON extras asserted; identical prompts yield identical `prompt_hash`.

2. Unpriced path: unknown model → warning + NULL cost + tokens present; then `prices set` → `prices backfill` → cost filled, snapshot written.
3. Normalizers: OpenAI cached-token subtraction; Anthropic disjoint mapping — against realistic usage-payload fixtures.
4. `tags()`: nesting overrides; isolation across concurrent asyncio tasks.
5. Never-raise: recorder given a broken DB session still returns the model response; failure is logged.

Exit criteria: install/lint/type/tests green; demo (script or README snippet): mock client makes a few tagged calls → `ledgerlm summary --by project` prints sensible totals including the unpriced count. **Stop for review.**

Phase 1.5 — Streaming capture · HARD GATE 2

Goal: streamed calls are recorded as faithfully as non-streamed ones, without altering caller-visible behavior.

Build:

- Migration: add `first_token_ms INTEGER NULL` to `llm_events`.
- **OpenAI chat completions streaming:** if the caller did not set `stream_options.include_usage`, inject it, then intercept and swallow the final usage-only chunk so the caller-visible stream is byte-identical to what they wrote code against; if the caller set it themselves, record from it and pass it through. Responses API streaming: usage from the terminal completed event.
- **Anthropic streaming:** both the raw `stream=True` event iterator and the `messages.stream()` helper. Input-side usage from `message_start`; output/cumulative usage from the final `message_delta`.
- Sync and async variants for both providers. Record `latency_ms` = time to stream completion and `first_token_ms` = time to first content event.
- **Abandoned streams:** if a wrapped stream is closed/exited before completion, record the event with `status="error"`, `error_type="stream_abandoned"`, and whatever usage is known (e.g., Anthropic input tokens from `message_start`).
- Mock provider grows a streaming mode to drive all of this without network.

Required tests: streamed vs non-streamed identical requests (mock) produce identical usage and identical cost; injected usage-chunk swallowing verified (caller never sees a chunk with empty choices it didn't opt into); abandoned-stream row recorded; async streaming covered; env-gated live-key integration tests for both SDKs (skipped by default).

Exit criteria: tests green; a mock streamed call and its non-streamed twin show the same `cost_usd` in `ledgerlm summary`. **Stop for review.**

Phase 2 — Dashboard · HARD GATE 3

Goal: browse the ledger locally: where money goes, over time, by dimension — fully offline.

Build:

- FastAPI app factory + read-only `queries.py` (all aggregation SQL lives here; SQLite `json_each` vs Postgres JSON operators isolated behind this module).
- **Pages:** Overview (headline totals, spend/day line, spend by provider and by model, persistent unpriced-rows banner when > 0); Attribution (group-by selector over the five promoted columns *and* arbitrary `tags` keys; table + bar chart); Top calls (N most expensive, filterable, showing tags, tokens, latency, hash); Prices (model_prices with `last_verified` staleness hints).
- **Filters** (date-range presets, provider, model, project) as HTMX partial swaps; charts re-render from JSON fragments.
- Assets vendored into `static/` (htmx.min.js, chart.umd.js, style.css) — zero external requests.
- `ledgerlm dashboard [--host 127.0.0.1] [--port 8642]`; localhost-only default documented as the v0 security model (no auth).
- `ledgerlm dev seed-demo`: generate a synthetic ~100k-row ledger (several projects/features/models, cache usage, some unpriced, some errors) for development, screenshots, and performance checks.
- Dockerfile + docker-compose (dashboard service, `./data` volume), with the macOS/Windows WAL bind-mount caveat documented; host-run remains the primary mode.

Required tests: query-layer unit tests against a seeded fixture DB (totals, group-bys incl. JSON-tag group-by, top-N, unpriced counts); route smoke tests via FastAPI TestClient (200s + key numbers present in HTML).

Exit criteria: dashboard renders the real ledger with the network panel showing zero non-localhost requests; every page responds in well under a second against the 100k-row seed; `docker compose up` works on a Linux host. **Stop for review.**

Phase 3 — Alerts, optimizer, export · HARD GATE 4

Goal: the ledger starts talking back: spikes surface within a day, savings candidates are quantified honestly, data exits cleanly.

Build:

- **Alert config** in `ledgerlm.toml`: `[alerts]` with `daily_budget_usd`, `spike_multiplier` (default 2.0), `baseline_days` (default 7), `min_spend_floor_usd` (default 1.00, anti-noise), `webhook_url`, optional `webhook_secret` (sent as a header), `cooldown_minutes` (default 360).
- **Rules**: *budget* — current UTC-day spend $\geq$ `daily_budget_usd`; *spike* — trailing-24h spend $\geq$ `spike_multiplier` $\times$ median of the prior `baseline_days` daily totals, and $\geq$ the floor.
- **Evaluation surfaces**: `ledgerlm alerts check` (cron-able, exit code reflects firing) and an optional background tick inside the dashboard process — both through the same evaluation code path.
- **Delivery**: webhook POST JSON `{rule, window, observed, threshold, top_contributors[ $\leq$ 5 by project/model]}` via httpx; firings persisted to `alert_firings`; cooldown dedupe per rule.
- **Optimizer** (`ledgerlm optimize` + dashboard page), three arithmetic-only analyses, each carrying the disclaimer *"repricing is arithmetic on identical tokens — it is not a claim that output quality would match; treat as candidates for experiments"*: (a) what-if repricing: for each (project|feature $\times$ model) group, recompute the group's exact token buckets at other models' current rates $\rightarrow$ "identical tokens on X = \$Y (–Z%)"; (b) token-heavy calls: calls above the p95 input-token count within their feature; (c) cache candidates: `prompt_hash` groups with $\geq$ N repeats and zero `cache_read_tokens`, with estimated savings at cache-read rates where priced.
- **Export**: `ledgerlm export events|summary --since ... --format csv [--out path]`.

Required tests: forced spike on a fixture ledger fires exactly one webhook (httpx mocked), then respects cooldown; budget rule boundary conditions; optimizer outputs asserted against hand-computed fixtures; CSV round-trips through a reader.

Exit criteria: on a synthetic spike, `alerts check` fires once and only once per cooldown window; optimizer report renders in CLI and dashboard with disclaimers; export opens in a spreadsheet. **Stop for review.**

Phase 4 — Hardening & release v0.1.0 (finish) · HARD GATE 5 = ship

Goal: the tool has proven itself on its own author's workload, and anyone can `pip install` it.

Build / do:

- **Dogfood:** instrument the blog-automation network for $\geq$ 1 week of real traffic.
- **Reconciliation (the honesty test):** compare LedgerLM's recorded totals for that window against the Anthropic and OpenAI console/usage reports for the same window. Investigate every delta until explained (unpriced rows, pre-1.5 streamed calls, timed-out

requests, price drift) and write **RECONCILIATION.md** documenting method, numbers, and causes. Target: within a few percent, itemized.

- Fix what dogfooding surfaces; this is the phase's real backlog.
- Docs: README with real screenshots (from **seed-demo**), full quickstart, CHANGELOG; version to 0.1.0.
- Packaging: confirm PyPI name availability (fallbacks decided at the gate if taken); build sdist/wheel; TestPyPI → clean-venv install smoke → publish to PyPI; tag **v0.1.0**.
- Launch: Bench Notes post drafted from the real dogfood + reconciliation data (written outside this repo).

Exit criteria: reconciliation within stated tolerance with causes itemized; **pip install ledgerlm** works in a clean venv end-to-end (init → wrap → summary); **v0.1.0** tagged and published. **v0 is done.**

10. Post-v0 parking lot

Each deferred item, and the v0 seam it plugs into — the point of listing these is to keep v0 from precluding them:

Item	Plugs into
Local proxy mode (OpenAI-compatible; language-agnostic)	Second producer feeding the recorder (§5)
More providers (Gemini, Bedrock, ...)	One new adapter in providers/
Batch API ingestion	New ingestion path writing llm_events (importer, not a wrapper)
Invoice reconciliation as a feature	New table + dashboard view; manual process proven in Phase 4
Slack/Discord alert delivery	Delivery plugins beside the webhook sender
Hard budget caps / enforcement	Opt-in gateway enforcement — note the tension with principle 4 (never block); default must remain never-block
Hosted multi-tenant SaaS, auth, seats	Recorder behind an API service + Postgres; auth layer above the dashboard

11. Risks and mitigations

Price staleness is the biggest threat to the product's one promise. Mitigations:

`price_snapshot` on every row (history never silently shifts), `last_verified` surfaced in the dashboard, unpriced counts on every cost surface, `prices backfill`, and the Phase 4 reconciliation gate.

SQLite write contention from many concurrent agents: WAL + `busy_timeout` handle realistic single-user volume; the Postgres path exists if it ever doesn't. **Streaming edge cases** (abandoned streams, injected-usage-chunk visibility) get dedicated tests in 1.5 and an explicit `stream_abandoned` status rather than silent gaps. **Docker Desktop WAL flakiness** is documented, with host-run as the primary mode. **Optimizer credibility**: arithmetic-only claims with mandatory disclaimers (principle 9). **Provider usage-schema drift**: `raw_usage` is stored verbatim, so even a broken normalizer loses nothing permanently; adapter tests pin fixture payloads. **LiteLLM ships something adjacent**: mitigation is speed, the single-user local niche, and an accuracy-first brand they aren't positioned for.

12. Working agreement (how we build)

One phase per Claude Code run. The first message of every session is:

Read DESIGN.md and CLAUDE.md. Execute Phase N only. Stop at the gate.

At a hard gate, the session stops and produces: what was built, how each exit criterion was verified, and every deviation from this document. The human reviews (and actually runs the demo). Deviations that survive review become Decision Log rows and, where needed, edits to this document — committed like code. `CLAUDE.md` carries the current-phase marker and any in-flight notes between sessions. Tests never touch the network; live-key integration tests are always env-gated. Small conventional commits throughout.

Recurring procedures live in the **project skill harness** at `.claude/skills/`, committed with the repo: `executing-a-phase` (the gate workflow itself), `building-provider-adapters` (the normalization contract), `changing-the-schema` (portable migrations), and `computing-costs` (money-math invariants). The split is deliberate: `CLAUDE.md` holds always-on conventions; skills hold on-demand procedure that loads when the matching work appears. The harness is contract, like this document — if gate review shows a skill gave wrong or missing guidance, the skill is fixed in the same commit wave as the code, and the harness may grow new skills at gates when a procedure proves recurring.

13. Decision log

#	Decision	Rationale (compressed)
D1	Instrument via <code>wrap()</code> client proxy, not a local proxy server	Zero migration for direct-SDK codebases; full SDK typing; nothing new in the request path that can fail. Proxy mode = post-v0 second producer.
D2	Sync + async from day one; streaming capture in Phase 1.5	SDKs mirror sync/async surfaces (cheap now, rewrite later); cost math is stream-agnostic, so capture can follow one gate behind.
D3	Cache-aware pricing from day one (4 disjoint token buckets)	OpenAI caches automatically — cache-blind math is wrong on day one; retrofitting loses historical breakdowns permanently.
D4	Unknown price → <code>cost_usd = NULL</code> ("unpriced"), never \$0; <code>prices backfill</code>	\$0 is indistinguishable from free; NULL keeps totals honest and repairable.
D5	No retry layer; one event per completed call	Both SDKs retry internally; LedgerLM stays out of the request path.
D6	Default ledger is per-user (<code>~/.ledgerlm/ledgerlm.db</code>), env-overridable	Multiple apps/agents attribute into one ledger by default.
D7	Dashboard = FastAPI + Jinja2 + HTMX + Chart.js, vendored	One language, no JS toolchain for a solo backend maintainer; JSON endpoints preserve the SPA option; vendoring honors offline.
D8	Store <code>prompt_hash</code> (SHA-256), never prompt content	Enables repeat-prompt/cache detection with nothing sensitive at rest.

#	Decision	Rationale (compressed)
D9	Tags = 5 promoted indexed columns + JSON extras	Fast, obvious queries on the dimensions that matter; unlimited flexibility for the rest.
D10	Aggregate on read; no rollup tables in v0	SQLite handles these queries at single-user scale; add rollups only on measured need.
D11	Alert config in TOML file; only firings in DB	Config is local-first and versionable; DB state only where dedupe requires it.
D12	Streaming: auto-inject <code>include_usage</code> and swallow the extra chunk	Faithful recording with byte-identical caller-visible streams.
D13	Project skill harness pre-seeded in <code>.claude/skills/</code> (executing-a-phase, building-provider-adapters, changing-the-schema, computing-costs)	CLAUDE.md carries always-on conventions; skills carry on-demand procedure, so every session runs phases, adapters, migrations, and money math the same way. Built fresh for this project (prior-project harnesses aren't visible from here); evolves at gates like this document.

New rows are appended at gates as deviations are accepted.